HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

CAA - Mini-projet

Ransomware post-quantique

Dylan Oliveira Ramos

18-12-2025

1. Utilisation de l'IA

Ce projet a été réalisé avec l'aide de l'IA, notamment pour la rédaction de la documentation et la génération de certaines parties du code source. Néanmoins, l'architecture cryptographique et le choix des algorithmes ont été conçus par mes soins.

2. Niveau de sécurité choisi

Le ransomware utilise le niveau de sécurité **V**, qui offre une sécurité au moins aussi forte que AES-256.

3. Description du ransomware

Le ransomware est composé d'un client (ordinateur de la victime) et d'un serveur (contrôlé par l'attaquant). Pour des raisons de simplicité, les deux entités sont exécutées dans le même programme.

Lors du démarrage de celui-ci, les options sont les suivantes :

1. `Encrypt files` : pour chiffrer tous les fichiers d'un dossier choisi.
2. `Pay ransom` : pour payer la rançon et pouvoir déchiffrer tous les fichiers.
3. `Decrypt one file` : pour déchiffrer un fichier spécifique et payer une plus petite rançon.
4. `Change master password` : pour changer le mot de passe utilisé pour tout déchiffrer.

4. Gestion des clés

4.1. Clés asymétriques

Le programme utilise deux paires de clés asymétriques, le serveur possède les clés privées et le client les clés publiques. La première paire est utilisée pour établir le secret partagé entre le client et le serveur avec l'algorithme **Kyber-1024**. La deuxième paire est utilisée pour signer les messages envoyés par le serveur avec l'algorithme **Dilithium 5**.

Ces deux algorithmes ont été choisis car ils sont post-quantiques, ont un bon compromis taille/sécurité et offrent le niveau de sécurité **V** défini précédemment. La taille des clés de **Kyber-1024** est de 1568 bytes pour la clé publique et 3168 bytes pour la clé privée. La taille des clés de **Dilithium 5** est de 2592 bytes pour la clé publique et 4864 bytes pour la clé privée, les signatures ont une taille de 4595 bytes.

4.2. Clés symétriques

Il y a quatre types de clés symétriques utilisées dans le programme :

- `Communication Key` : clé dérivée du secret partagé avec **HKDF**, utilisée pour chiffrer les communications entre le client et le serveur avec **AES-256-GCM**.
- `Master Key` : clé dérivée avec **Argon2id** à partir d'un mot de passe aléatoire d'un dictionnaire (`Master Password`).
- `Root Key` : clé générée de manière aléatoire, utilisée pour chiffrer les `File Key` avec **AES-256-GCM**.
- `File Key` : clé dérivée avec **Argon2id** à partir d'un mot de passe aléatoire du dictionnaire, unique pour chaque fichier, utilisée pour chiffrer le fichier avec **AES-256-GCM**.

La `Communication Key` est dérivée avec **HKDF** car l'algorithme est conçu pour dériver des clés à partir de secrets partagés avec une bonne entropie. Cette dérivation s'effectue avec les paramètres suivants :

- Algorithme de hachage : SHA-256.

- Taille de la clé dérivée : 32 bytes (pour être compatible avec AES-256).
- Sel : aucun (RFC 5869).

La `Master Key` et les `File Key` sont dérivées avec **Argon2id** car l'algorithme est conçu pour dériver des clés à partir d'entrées à entropie faible comme des mots de passe, ce qui permet à l'utilisateur de déchiffrer ses fichiers en entrant simplement un mot de passe. Cette dérivation s'effectue avec les paramètres suivants (paramètres par défaut, à adapter selon la machine) :
- Taille du sel : 16 bytes.
- Taille de la clé dérivée : 32 bytes (pour être compatible avec AES-256).
- Nombre d'itérations : 1.
- Degré de parallélisme : 4.
- Coût en mémoire : 65536 KB.

Concernant `AES-256-GCM`, la taille de clé de 32 bytes (256 bits) a été choisie pour correspondre au niveau de sécurité **V** défini précédemment. Les paramètres suivants sont utilisés (paramètres recommandés) :
- Taille de la clé : 32 bytes.
- Taille du nonce : 12 bytes.
- Taille du tag : 16 bytes.

Cela permet de chiffrer des fichiers d'une taille maximale d'environ **68 GB**.

5. Communication entre le client et le serveur

Le mécanisme d'échange de clés post-quantiques **ML-KEM** (Kyber) permet de se protéger contre les attaques « harvest now, decrypt later » en garantissant que même si un attaquant enregistre les communications aujourd'hui, il ne pourra pas les déchiffrer à l'avenir avec un ordinateur quantique. C'est pour cette raison que le ransomware utilise cet algorithme pour établir un secret partagé entre le client et le serveur afin d'en dériver une clé de communication symétrique d'une taille de 256 bits, qui elle, n'est pas vulnérable aux attaques quantiques.

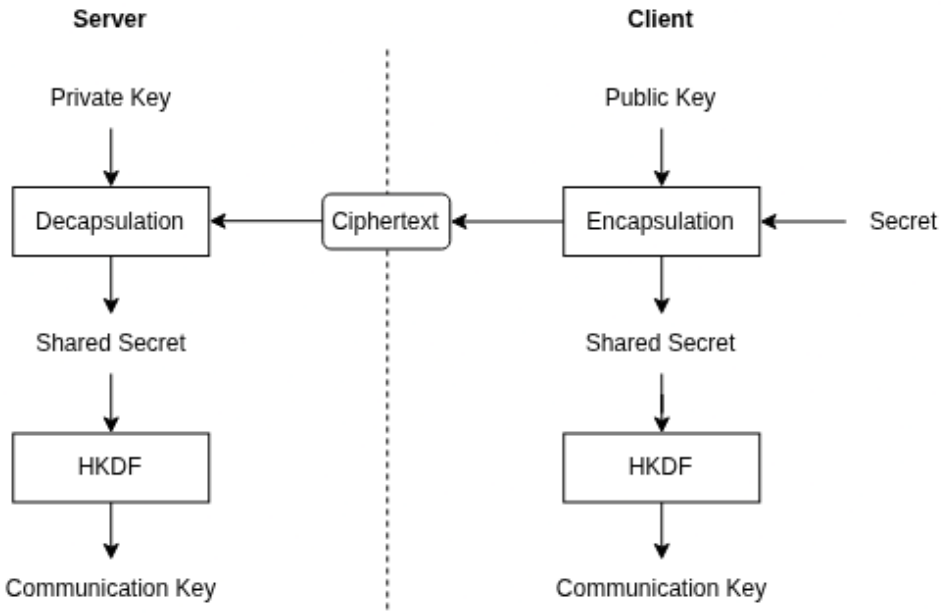

Fig. 1. – Établissement de la clé symétrique pour la communication sécurisée entre le client et le serveur.

6. Chiffrement des fichiers

Chaque fichier est chiffré avec sa `File Key` respective. Cette `File Key` est ensuite chiffrée avec la `Root Key`, qui est elle-même chiffrée avec la `Master Key`.

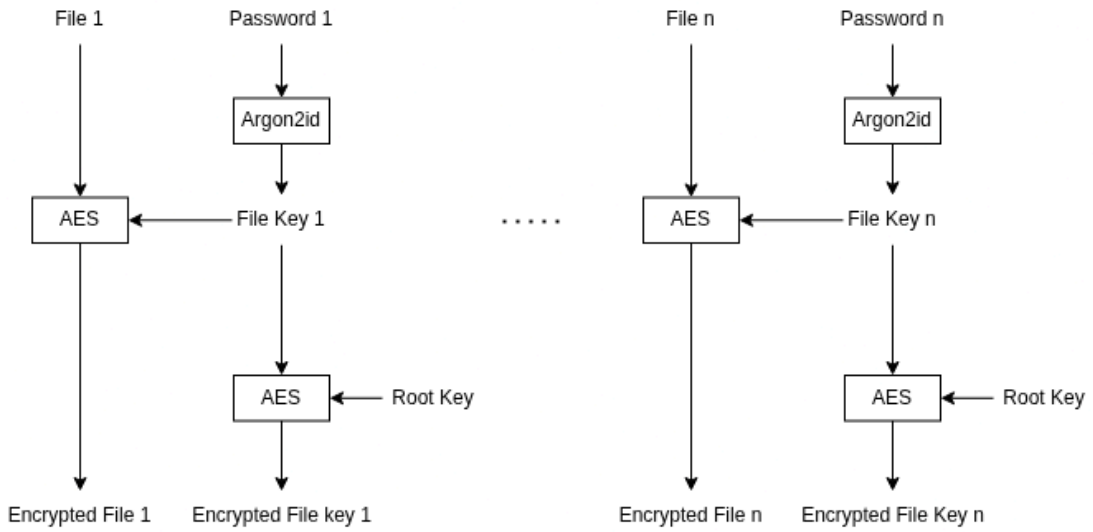

Fig. 2. – Chiffrement des fichiers et des clés de fichier sur le client.

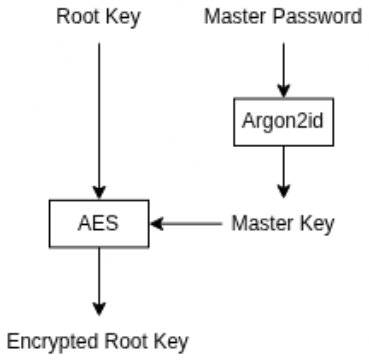

Fig. 3. – Chiffrement de la `Root Key` sur le client.

À la fin du chiffrement, tous les mots de passe utilisés pour dériver les clés sont envoyés au serveur et supprimés du client.

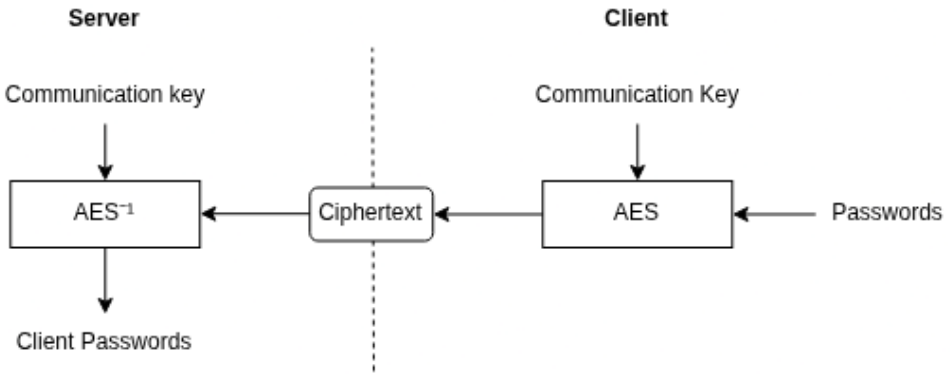

Fig. 4. – Envoi des mots de passe au serveur.

6.1. Stockage des métadonnées

Lors du chiffrement d'un fichier, son contenu est remplacé par le contenu concaténé suivant :

File ID	Password Salt	File Key IV	File Key Tag	File Key Ciphertext	File IV	File Tag	File Ciphertext

Le stockage de ces données permet de déchiffrer le fichier ultérieurement de deux manières :

- Avec le mot de passe du fichier (utilisé avec le sel stocké pour dériver la `File Key`), dans le cas où l'utilisateur paie la rançon pour déchiffrer un seul fichier.
- Avec la `Root Key` (utilisée pour déchiffrer la `File Key`), dans le cas où l'utilisateur paie la rançon pour déchiffrer tous les fichiers.

Lorsque tous les fichiers ont été chiffrés, un fichier de métadonnées est créé à la racine du dossier, contenant les données concaténées suivantes :

File ID	Master Password Salt	Root Key IV	Root Key Tag	Root Key Ciphertext

Le stockage de ces données permet de déchiffrer la `Root Key` ultérieurement avec la `Master Key` (dérivée du `Master Password` et du sel).

Note : Le `File ID` 0 est réservé pour le fichier de métadonnées à la racine.

7. Déchiffrement des fichiers

Pour déchiffrer tous les fichiers, il faut connaître le `Master Password`, qui est envoyé par le serveur une fois la rançon payée. Celui-ci permet de dériver la `Master Key`, qui permet de déchiffrer la `Root Key`, qui permet de déchiffrer les `File Key` et enfin les fichiers.

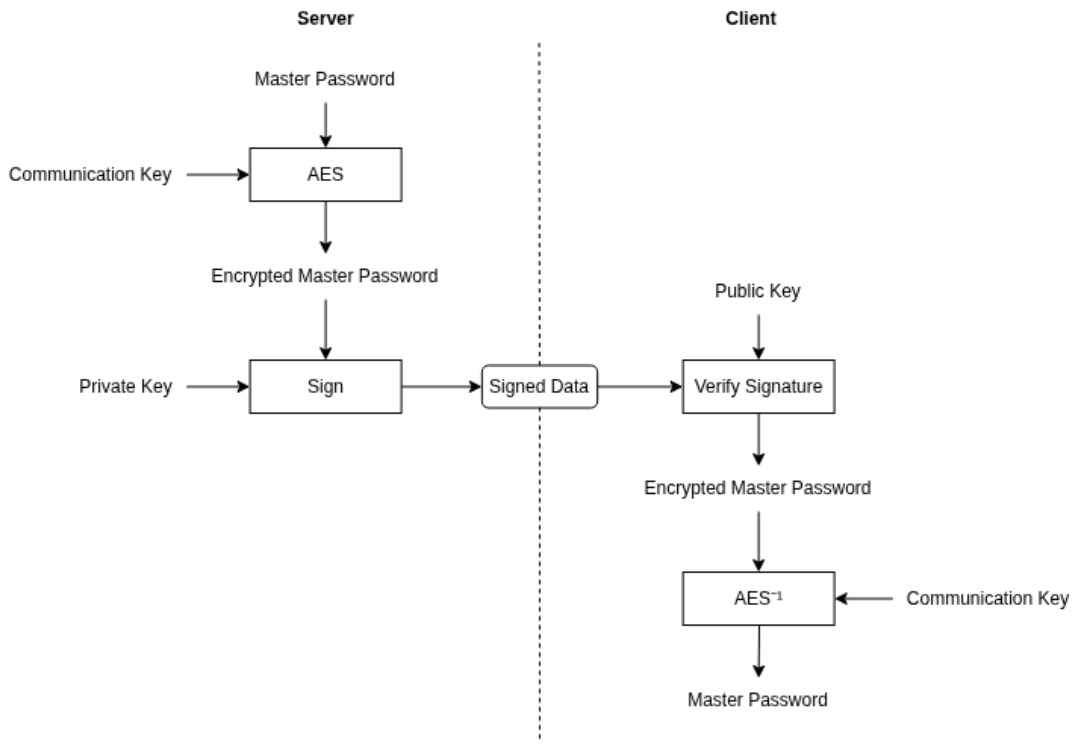

Fig. 5. – Envoi du `Master Password` au client.

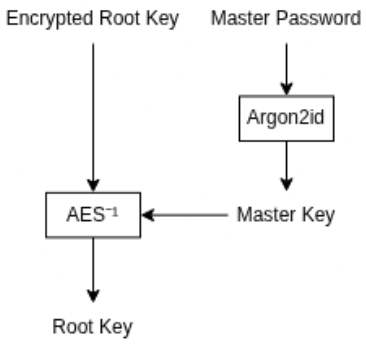

Fig. 6. – Déchiffrement de la `Root Key` sur le client.

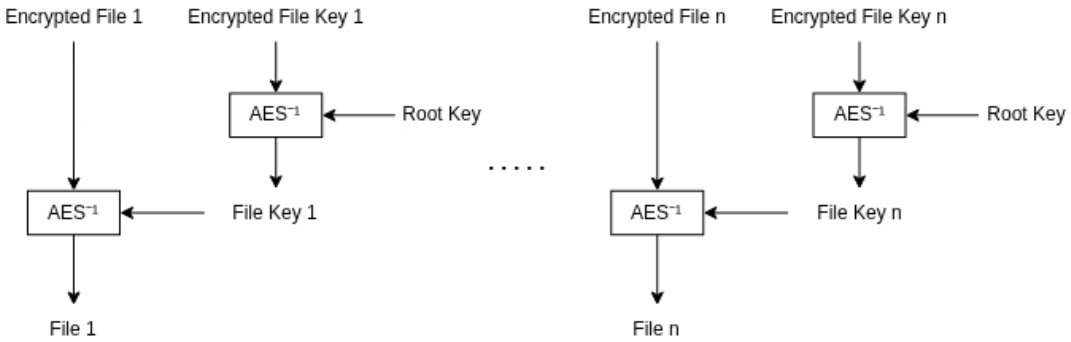

Fig. 7. – Déchiffrement des clés de fichier et des fichiers sur le client.

8. Déchiffrement d'un fichier spécifique

Pour déchiffrer un fichier spécifique, il faut connaître le mot de passe du fichier, qui est envoyé par le serveur une fois la rançon payée. Celui-ci permet de dériver la `File Key`, qui permet de déchiffrer le fichier. Pour cela, le client récupère l'identifiant du fichier choisi (stocké dans le fichier lui-même) puis l'envoie au serveur pour obtenir le mot de passe correspondant.

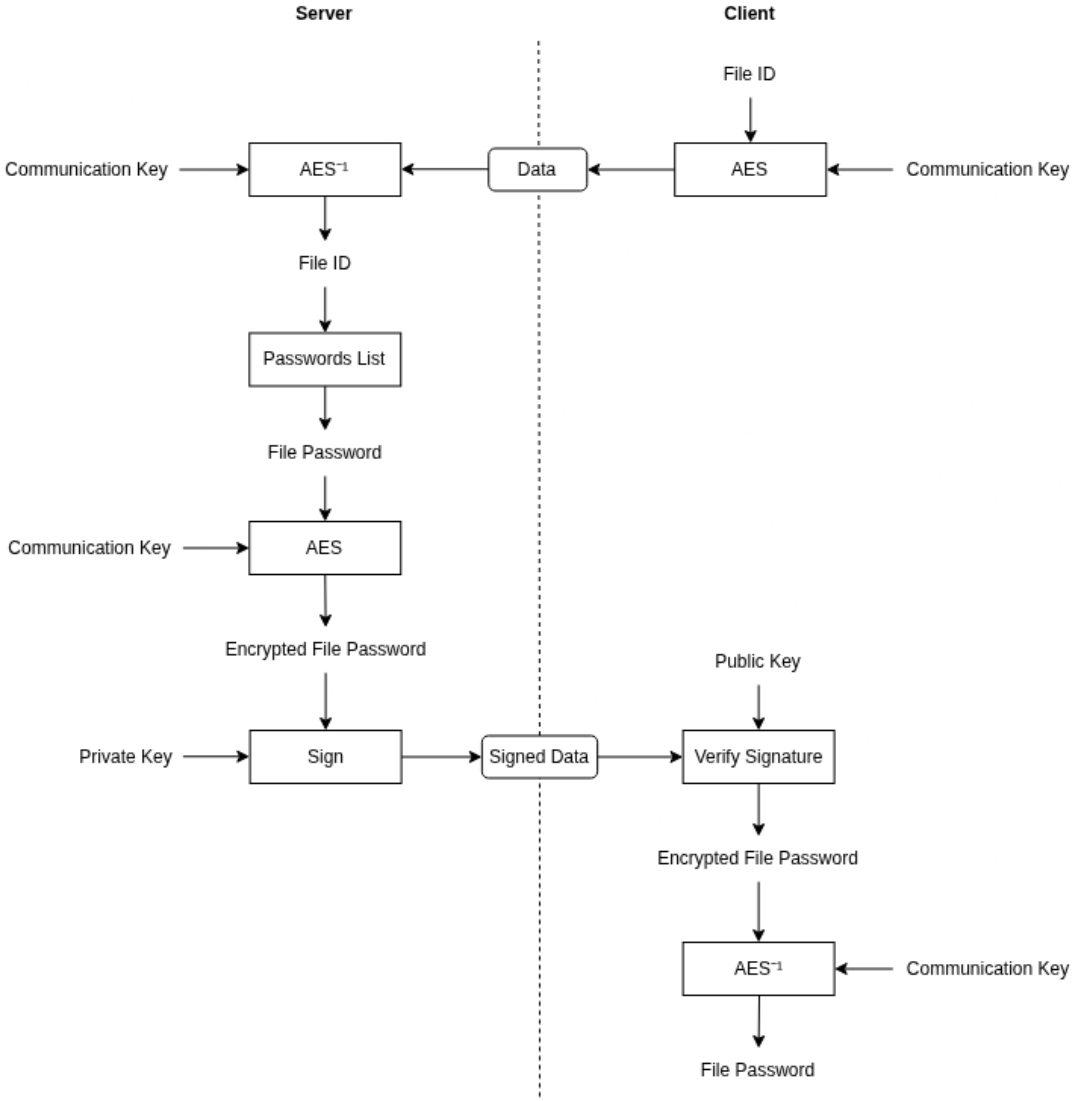

Fig. 8. – Envoi du mot de passe du fichier au client.

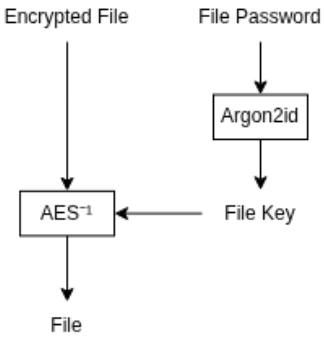

Fig. 9. – Déchiffrement d'un fichier spécifique sur le client.

9. Changement de mot de passe

Il est possible de changer le `Master Password` après le chiffrement des fichiers sans avoir à rechiffrer tous les fichiers. En effet, le `Master Password` n'est utilisé que pour chiffrer la `Root Key`. Ainsi, pour changer le mot de passe, il suffit de déchiffrer la `Root Key` en utilisant l'ancien mot de passe, puis de la rechiffrer avec le nouveau mot de passe.

Lors de la demande de changement, le client envoie les métadonnées (fichier créé à la racine du dossier) ainsi qu'un nouveau `Master Password` au serveur. Étant donné que le serveur connait le `Master Password` initial, il peut déchiffrer la `Root Key` et la rechiffrer avec le nouveau mot de passe. Ensuite, il renvoie les nouvelles métadonnées au client pour qu'il puisse les stocker.

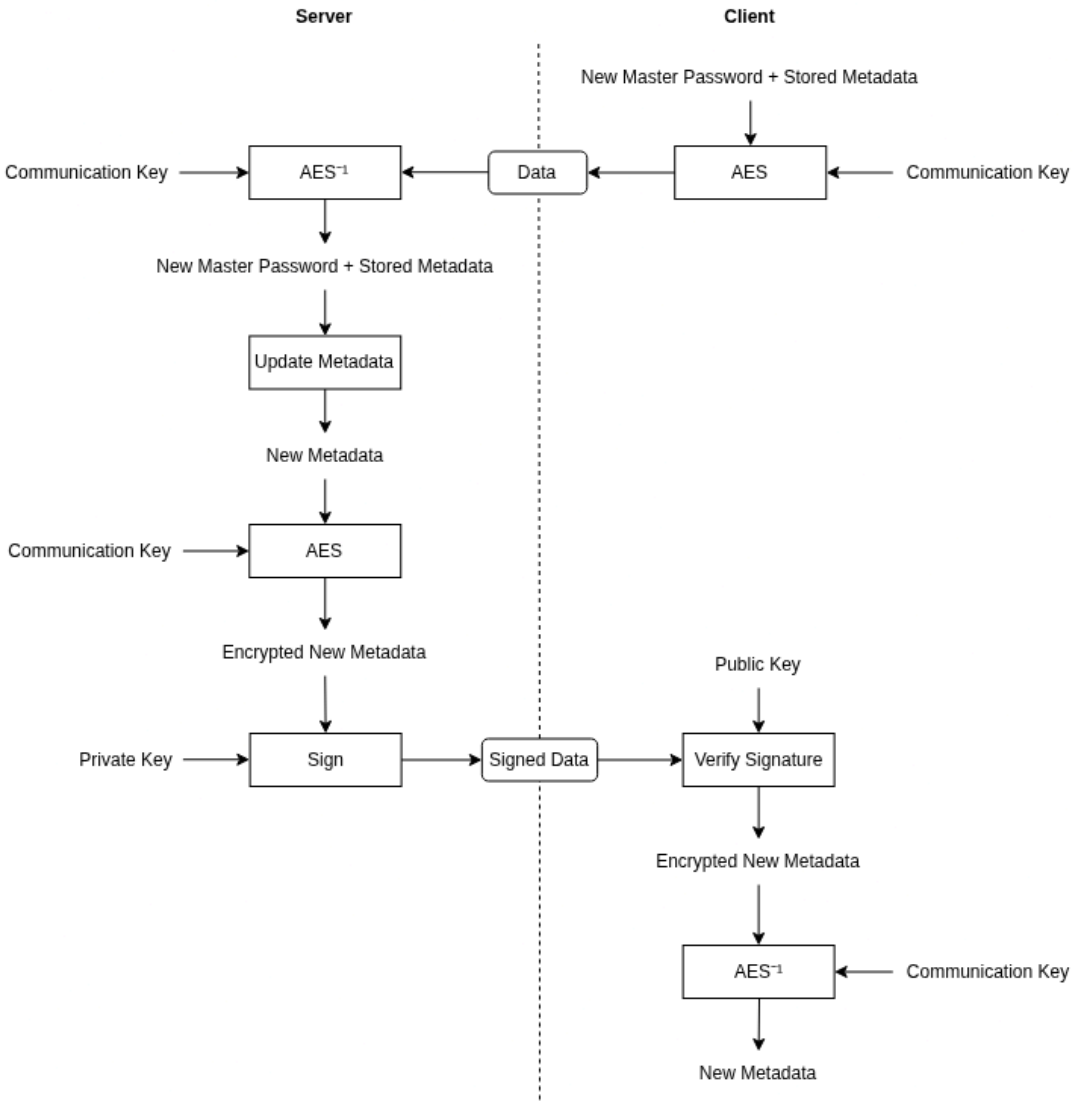

Fig. 10. – Changement du `Master Password` sur le serveur et mise à jour des métadonnées sur le client.

10. Légitimité des mots de passe envoyés par le serveur

Pour s'assurer que les mots de passe envoyés par le serveur sont légitimes et n'ont pas été modifiés par un attaquant, le serveur signe chaque message envoyé avec sa clé privée **ML-DSA** (Dilithium). Le client vérifie ensuite la signature avec la clé publique du serveur avant d'utiliser les mots de passe reçus.